

ASSIGNMENT - 5

Sub Code: CSA10

Subject Name: Software Engineering

No.	Questions	Marks	CO
1	Explain technical debt. What are the major causes of technical debt, and how can organizations minimize it during software evolution?	10	
2	Explain how AI/ML and Edge Computing are transforming modern software engineering. Provide one practical example for each.	10	
3	Case Study – A company deploys dozens of microservices in the cloud and notices that resource usage and energy consumption are increasing. Explain three sustainable DevOps practices that can reduce energy consumption and cloud cost.	5	

1.

Technical Debt

Technical debt refers to the extra effort, cost, and complexity that an organization must deal with in the future because of quick, temporary, or poor technical decisions made during software development. Similar to financial debt, technical debt may help a project move faster initially, but it creates additional work later.

Major Causes of Technical Debt

1. **Tight deadlines:** Developers may use quick solutions instead of designing a proper long-term solution.
2. **Poor system design:** Weak architecture can make the software difficult to maintain and extend.
3. **Lack of testing:** Insufficient testing can leave bugs and unstable code in the system.
4. **Outdated technologies:** Old libraries, frameworks, and dependencies can create compatibility and security problems.
5. **Code duplication:** Repeated code increases maintenance effort.
6. **Lack of documentation:** Poor documentation makes it difficult for new developers to understand and modify the system.
7. **Frequent requirement changes:** Continuous changes may result in temporary fixes and inconsistent code.
8. **Lack of refactoring:** Existing code may become complex when it is not regularly improved.
9. **Inexperienced development practices:** Poor coding standards and architectural decisions can increase debt.

10. **Insufficient code review:** Without proper review, inefficient or poor-quality code may enter the project.

How Organizations Can Minimize Technical Debt

- Follow coding standards and best practices.
- Perform regular code reviews.
- Conduct refactoring regularly.
- Use automated testing and CI/CD pipelines.
- Keep dependencies and technologies updated.
- Maintain proper technical documentation.
- Use good software architecture and design principles.
- Track technical debt as part of project planning.
- Allocate dedicated time for fixing technical debt.
- Use monitoring and code-quality tools to identify problems early.

Conclusion:

Technical debt cannot always be completely avoided, but organizations can control it through good design, testing, documentation, code reviews, and continuous refactoring. Managing technical debt improves software quality, maintainability, security, and long-term development speed.

2. AI/ML and Edge Computing in Modern Software Engineering

A. AI/ML in Software Engineering

Artificial Intelligence (AI) and Machine Learning (ML) are transforming software engineering by automating development tasks, analyzing large amounts of data, detecting problems, and improving decision-making.

Applications of AI/ML

- Automated code generation and code completion.
- Bug and vulnerability detection.
- Predictive maintenance.
- Automated software testing.
- Performance prediction and optimization.
- Intelligent monitoring and anomaly detection.
- Personalized software applications.

Practical Example:

A software company can use an ML model to analyze historical application logs and predict when a server or service is likely to fail. The system can alert the DevOps team before the failure occurs, allowing preventive action.

B. Edge Computing

Edge computing processes data close to where it is generated instead of sending all data to a centralized cloud server. This reduces network latency and bandwidth usage.

Advantages of Edge Computing

- Faster response time.
- Reduced network traffic.
- Lower cloud processing costs.
- Better performance for real-time applications.
- Improved reliability when internet connectivity is limited.
- Can improve data privacy by processing sensitive data locally.

Practical Example:

In a smart manufacturing factory, sensors can detect abnormal machine vibrations. An edge device can analyze the sensor data locally and immediately stop the machine if a dangerous condition is detected, instead of waiting for the data to travel to the cloud.

Conclusion

AI/ML makes software development and operation more intelligent and automated, while edge computing makes applications faster and more responsive by processing data closer to users and devices. Both technologies are important parts of modern software engineering.

3. Case Study – Sustainable DevOps Practices

A company running dozens of cloud-based microservices may experience increasing resource usage and energy consumption. Sustainable DevOps, also called Green DevOps, can reduce both environmental impact and cloud costs.

1. Resource Optimization and Right-Sizing

Organizations should monitor CPU, memory, storage, and network usage and allocate only the resources actually required.

For example, if a service uses only 30% of an allocated CPU capacity, its resources can be reduced. Unused virtual machines and containers can also be stopped or removed.

Benefits:

- Reduces unnecessary computing.
- Lowers cloud costs.

- Reduces energy consumption.

2. Auto-Scaling and Efficient Container Management

Microservices can use containers and auto-scaling to automatically increase resources during high demand and reduce them during low demand.

For example, during peak hours, more containers can be started to handle traffic. At night, unnecessary containers can be stopped.

Benefits:

- Avoids running unused resources.
- Improves resource utilization.
- Reduces energy consumption and cloud bills.

3. Efficient CI/CD and Workload Scheduling

DevOps teams can optimize CI/CD pipelines by avoiding unnecessary builds and tests and scheduling non-urgent workloads during periods when infrastructure is less heavily utilized. Build caching and reusable artifacts can also reduce repeated computation.

Benefits:

- Reduces CPU and compute usage.
- Makes pipelines faster.
- Decreases cloud resource consumption.
- Reduces operational cost.

Conclusion

Sustainable DevOps combines resource optimization, intelligent auto-scaling, and efficient CI/CD practices to reduce unnecessary cloud computing. These practices help organizations lower energy consumption, carbon footprint, and cloud costs while maintaining application performance and reliability.